Project – High Level Design

CI/CD for Vue.js Technology App

Course Name: Devops Foundation

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Pranjal Mahwar	EN22CS301729
2.	Prateeksha Shukla	EN22CS301740
3.	Parthiv Prajapat	EN22EL301039
4.	Raj Patel	EN22CS301781
5.	Rishabh Dev Rajput	EN22CS17803

Group Name:06D7

Project Number:D0-29

Industry Mentor Name: Aashruti Shah

University Mentor Name: Shyam Patel

Academic Year:2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

This document provides a detailed overview of the design and implementation of a CI/CD pipeline for a Vue.js application using modern DevOps practices. The system leverages cloud infrastructure provided by Amazon Web Services, specifically Amazon EC2 instances, to ensure scalability, reliability, and ease of deployment.

The project integrates tools such as Git, Docker, GitHub Actions/Jenkins, and Kubernetes to automate the software delivery lifecycle, from code integration to deployment.

1.1 Scope of the Document

This document covers:

- Design and setup of CI/CD pipeline for a Vue.js application
- Configuration of cloud infrastructure using AWS EC2 instances
- Integration of version control (Git) with CI/CD tools (GitHub Actions/Jenkins)
- Containerization using Docker
- Deployment using Kubernetes (K8s)
- Automation of build, test, and deployment processes

It provides both conceptual understanding and practical implementation details required for the project.

1.2 Intended Audience

This document is intended for:

- Students and instructors involved in the DevOps project
- Developers working on Vue.js applications
- DevOps engineers and beginners learning CI/CD pipelines
- System administrators managing cloud-based deployments

1.3 System Overview

The system is designed as a fully automated CI/CD pipeline deployed on AWS infrastructure. The workflow is as follows:

1. **Code Development**
   Developers write and push code to a Git repository (e.g., GitHub).
2. **Continuous Integration (CI)**
   - Triggered on every commit
   - Automated build and unit testing using GitHub Actions or Jenkins
   - Ensures code quality and early bug detection
3. **Containerization**
   - Application is packaged into Docker containers
   - Images are stored in a container registry
4. **Cloud Infrastructure (AWS EC2)**
   - EC2 instances act as servers for hosting the application
   - Provides scalable and flexible compute resources
5. **Continuous Deployment (CD)**
   - On successful build, the application is deployed automatically
   - Kubernetes manages container orchestration and scaling
6. **Monitoring & Reliability**
   - Ensures high availability and fast deployment cycles
   - Supports enterprise-level application performance

This architecture ensures a streamlined development lifecycle with minimal manual intervention, faster releases, and improved software quality.

2.1 Application Design

The application design focuses on how the Vue.js application is structured, built, and deployed within the CI/CD pipeline.

1. Frontend Application (Vue.js)

- The application is developed using Vue.js framework
- Follows component-based architecture
- Organized into:
    - Components (UI elements)
    - Views (pages)
    - Services (API calls)
    - Assets (CSS, images)

2. Version Control Integration

- Source code is stored in GitHub repository
- Developers push changes to branches (main/develop)
- Each push triggers the CI/CD pipeline automatically

3. Build Process

- CI tool (GitHub Actions/Jenkins) installs dependencies
- Runs build command:

```
npm install
npm run build
```

- Generates optimized production-ready files

4. Containerization using Docker

- Application is packaged into a Docker image
- A Dockerfile defines:
  - Base image (Node/Nginx)
  - Build steps
  - Deployment configuration
- Ensures consistency across environments

5. Deployment Architecture

- Docker container is deployed on Amazon EC2
- Kubernetes manages:
  - Pod creation
  - Scaling
  - Load balancing

6. CI/CD Workflow Integration

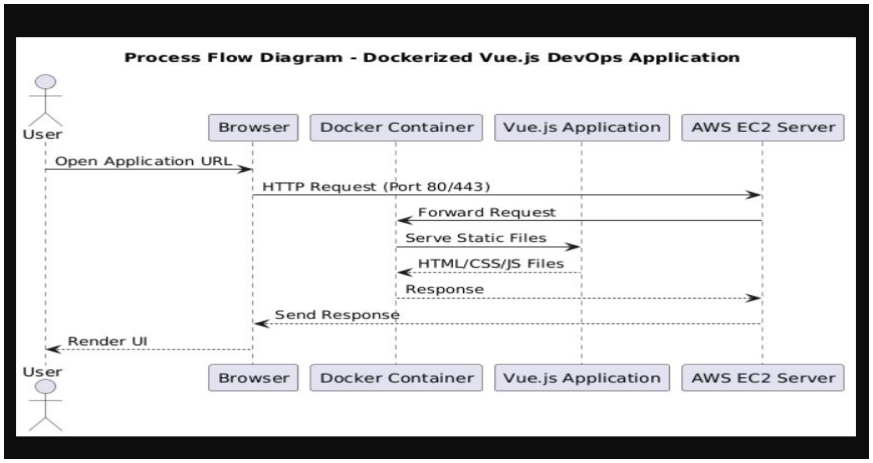
- **CI Phase:**
  - Code push triggers pipeline
  - Build + test execution
- **CD Phase:**
  - Docker image creation
  - Deployment to EC2 via Kubernetes

7. Key Features of Application Design

- Modular and reusable components
- Automated build and deployment
- Scalable cloud-based hosting
- High availability and reliability
- Easy rollback and updates

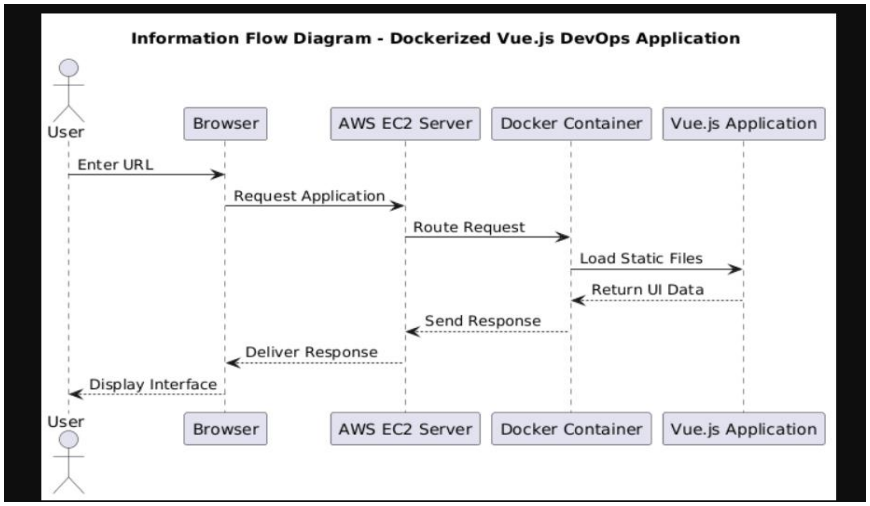
2.2 Process Flow Description:

The process flow represents the sequence of operations performed when a user accesses the application.



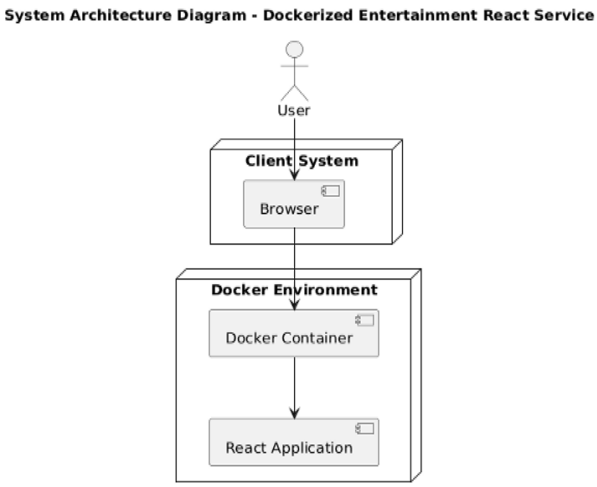
2.3 Information Flow :

The information flow describes how data is transferred between different components of the system.



2.4 System Architecture :

The architecture diagram provides a high-level representation of system components and their relationships.



2.5 Key Design Considerations

The system design is developed by considering several important factors to ensure efficiency, scalability, and reliability of the application deployed on Amazon Web Services using Amazon EC2.

1. Scalability

- The system is designed to scale horizontally using containerization and orchestration tools like Kubernetes
- AWS EC2 instances can be resized or replicated based on demand

2. Reliability

- Ensures continuous availability of the application
- Use of container-based deployment reduces failure impact
- Fault tolerance is achieved through multiple instances

3. Performance

- Optimized build process for Vue.js application
- Fast response time using lightweight containers
- Efficient resource utilization on EC2

4. Security

- Secure access using SSH keys for EC2
- Environment variables and secrets managed securely in CI/CD pipelines
- HTTPS support for secure communication

5. Automation

- Full CI/CD pipeline using GitHub Actions or Jenkins
- Automated build, test, and deployment processes
- Reduces manual errors and speeds up delivery

6. Maintainability

- Modular application structure (Vue components)
- Easy updates and rollback using Docker images

2.7 API Catalogue Current Implementation:

No APIs available (Static frontend application for now

3. Data Design

The data design for this project focuses on managing application artifacts, configuration data, logs, and deployment-related information within a CI/CD pipeline environment. Since the project is based on a **Vue.js application with DevOps practices**, the system primarily handles build artifacts, container images, and operational data rather than complex relational databases.

The data flow is structured to ensure consistency, reliability, and traceability across all stages of the CI/CD pipeline, from development to deployment on AWS EC2 instances.

3.1 Data Model

The data model in this system is relatively lightweight and is centered around the following key components:

- **Source Code Data**
  - Stored in a GitHub repository
  - Includes Vue.js application files, configuration files, and CI/CD workflow definitions
- **Build Artifacts**
  - Generated during the CI phase (e.g., compiled Vue.js files in /dist)
  - Stored temporarily during pipeline execution or packaged into Docker images
- **Container Images**
  - Docker images built from the application
  - Tagged with version identifiers (e.g., latest, commit hash)
  - Stored in a container registry such as Docker Hub
- **Deployment Configuration Data**
  - Includes Dockerfiles, Kubernetes YAML files (if used), and environment variables
  - Defines how the application is deployed on AWS EC2
- **Logs and Monitoring Data**
  - Build logs from CI/CD tools (GitHub Actions/Jenkins)
  - Runtime logs from Docker containers on EC2

This model ensures that all critical data required for building, deploying, and maintaining the application is properly organized and accessible.

3.2 Data Access Mechanism

The system uses secure and automated mechanisms to access and manage data throughout the pipeline:

- **Version Control Access**
  - Developers push and pull code using Git
  - GitHub acts as the central repository
- **CI/CD Pipeline Access**
  - GitHub Actions or Jenkins accesses the repository using secure tokens
  - Secrets (e.g., Docker credentials, SSH keys) are stored securely in environment variables
- **Container Registry Access**
  - Docker images are pushed and pulled using authenticated access (Docker Hub credentials)
- **AWS EC2 Access**
  - Deployment is performed using SSH access to EC2 instances
  - Secure key pairs (.pem files) are used for authentication
- **Application Access**
  - End users access the deployed application via the EC2 public IP or domain

This approach ensures secure, role-based, and automated access to all system data.

3.3 Data Retention Policies

To maintain efficiency and avoid unnecessary storage usage, the following data retention policies are applied:

- **Source Code**
  - Permanently stored in GitHub with version history
- **Build Artifacts**
  - Temporary storage during pipeline execution
  - Old artifacts are automatically cleaned up

- **Docker Images**
  - Only recent and stable versions are retained
  - Older or unused images are periodically deleted from Docker Hub
- **Logs**
  - CI/CD logs are retained for a limited duration (based on tool settings)
  - Application logs on EC2 may be rotated or deleted after a defined period
- **Backups**
  - Critical configurations and deployment files can be backed up if required

These policies help optimize storage and ensure system performance.

3.4 Data Migration

Data migration in this project refers to the movement of application data and artifacts across different stages of the pipeline:

- **Code Migration**
  - Developers push code changes from local systems to GitHub
- **Build Migration**
  - Source code is pulled into the CI environment for building and testing
- **Artifact Migration**
  - Built application is packaged into Docker images
- **Image Migration**
  - Docker images are pushed to Docker Hub and pulled onto AWS EC2 instances
- **Deployment Migration**
  - Updated containers replace older versions during deployment
- **Environment Migration (if applicable)**
  - Application can be moved between development, staging, and production environments

4. Interfaces

The system consists of multiple interfaces that enable communication between different components of the CI/CD pipeline, development environment, and deployment infrastructure on AWS EC2. These interfaces ensure smooth integration, automation, and data exchange across all stages of the software delivery lifecycle.

4.1 User Interface

The user interface of the system is the **Vue.js web application** deployed on AWS EC2.

- Accessible through a web browser using the EC2 public IP or domain
- Provides an interactive and responsive frontend for users
- Displays application data and features developed in the project
- Designed using modern UI components and frameworks within Vue.js

This is the primary interface through which end users interact with the system.

4.2 Developer Interface

The developer interface includes tools and platforms used by developers to write, manage, and push code.

- **Local Development Environment**
  - Tools like VS Code or any IDE
  - Node.js for running the Vue.js application locally
- **Version Control Interface**
  - Git commands (CLI) or GitHub web interface
  - Developers push code changes to the repository
- **Repository Hosting**
  - GitHub repository acts as the central interface for collaboration

4.3 CI/CD Interface

This interface handles automation of build, test, and deployment processes.

- **GitHub Actions / Jenkins Dashboard**
  - Displays pipeline status (success/failure)
  - Shows logs for each stage (build, test, deploy)
- **Workflow Configuration Files**
  - YAML files define pipeline steps
  - Automatically triggered on code push or merge
- **Integration Points**
  - Connects GitHub with Docker and AWS EC2
  - Uses secrets for secure authentication

This interface ensures continuous integration and delivery without manual intervention.

4.4 Containerization Interface

The containerization interface manages Docker-related operations.

- **Docker CLI / Docker Desktop**
  - Used to build and test container images locally
- **Docker Hub**
  - Stores container images
  - Acts as a registry for pushing and pulling images
- **Docker Engine on EC2**
  - Runs containers for deployment

This interface ensures application consistency across different environments.

4.5 Deployment Interface (AWS EC2)

This interface manages application deployment and hosting.

- **AWS EC2 Instance**
  - Hosts the application in a virtual server environment
- **SSH Access**
  - Secure login using key pairs (.pem files)
  - Used for remote deployment and management
- **Server Configuration**
  - Docker installed on EC2
  - Optional: Kubernetes setup for orchestration
- **Networking Interface**
  - Security groups to allow HTTP/HTTPS traffic
  - Public IP or domain for access

This interface is responsible for running the application in a live environment.

4.6External Service Interfaces

The system integrates with external services to support DevOps operations:

- **GitHub** → Source code management
- **Docker Hub** → Container image registry
- **AWS EC2** → Hosting and deployment
- **CI/CD Tools** → Automation (GitHub Actions / Jenkins)

These integrations ensure a complete and automated DevOps ecosystem.

5. State and Session Management

State and session management in this project ensures that the application behaves consistently for users and that system components in the CI/CD pipeline maintain proper execution context. Since the project is based on a Vue.js frontend deployed via Docker on AWS EC2, the focus is on managing frontend state, user sessions (if applicable), and pipeline execution states.

5.1 Application State Management

The application state refers to the data and UI state maintained within the Vue.js application.

- **Client-Side State**
  - Managed using Vue.js reactive data system or state management libraries like Vuex (if implemented)
  - Includes UI states such as form inputs, navigation states, and dynamic content
- **Component-Level State**
  - Each Vue component maintains its own local state
  - Used for handling component-specific logic and rendering
- **Global State (Optional)**
  - Shared data across multiple components (e.g., user info, settings)
  - Managed centrally to ensure consistency
- **Persistence**
  - Temporary state stored in browser memory
  - Optional use of Local Storage or Session Storage for persistence across reloads

This ensures a smooth and responsive user experience.

5.2 Session Management

Session management handles how user interactions are tracked during their usage of the application.

- **Session Handling**
  - If authentication is implemented, sessions are managed using tokens (e.g., JWT)
  - Tokens are stored securely in browser storage or cookies
- **Session Lifecycle**
  - Session starts when the user accesses or logs into the application
  - Session ends on logout, token expiry, or browser closure (for session storage)
- **Security Measures**
  - Use of HTTPS for secure communication
  - Token expiration and renewal mechanisms
  - Avoid storing sensitive data in plain text
- **Stateless Nature**
  - The deployed application is primarily stateless, making it scalable and easy to redeploy

5.3 CI/CD Pipeline State Management

The CI/CD pipeline also maintains state during execution:

- **Pipeline Execution State**
  - Each workflow run has a defined state (triggered, running, success, failed)
  - Managed by GitHub Actions or Jenkins

- **Stage-wise State**
  - Build → Test → Docker Build → Push → Deploy
- **Artifact State**
  - Intermediate artifacts (build files, Docker images) are passed between stages
- **Logging and Tracking**
  - Logs are generated at each step for monitoring and debugging
  - Enables traceability of deployments

5.4 Session Handling in Deployment Environment

- **Docker Containers**
  - Containers are stateless by design
  - Each deployment creates a fresh container instance
- **AWS EC2 Instance**
  - Maintains runtime environment but not application session data
  - Application sessions remain on the client side
- **Scaling Consideration**
  - Stateless design allows easy scaling (especially if Kubernetes is used)
  - No dependency on a single server for session storage

5.5 Data Consistency and Reliability

- Ensures consistent behavior across deployments
- Avoids dependency on server-side session storage

6. Caching

Caching is used to improve system performance by reducing repeated processing and data fetching.

- **Frontend Caching:**
  Browser stores static files (HTML, CSS, JS) to load the application faster. Local storage may store temporary data.
- **CI/CD Caching:**
  Dependencies (like node_modules) and build files are cached to reduce pipeline execution time.
- **Docker Caching:**
  Docker reuses unchanged layers during image builds, speeding up the build process.
- **Server-Side Caching:**
  Optional caching (e.g., Nginx, Redis) can be used on AWS EC2 to improve response time.
- **Cache Invalidation:**
  Versioning and cache expiry ensure users get updated content after deployment.

7. Non-Functional Requirements

Non-functional requirements define the quality attributes of the system, such as security, performance, scalability, and reliability. These requirements ensure that the CI/CD pipeline and deployed Vue.js application on AWS EC2 operate efficiently, securely, and consistently under different conditions.

7.1 Security Aspects

Security is a critical component of the system, as it involves protecting source code, credentials, application data, and infrastructure from unauthorized access and potential threats.

1. Authentication and Authorization

- Secure authentication mechanisms are used across all system components.
- Access to the AWS EC2 instance is controlled using **SSH key pairs (.pem files)** instead of passwords.
- GitHub repositories are secured with user authentication and access permissions.
- Role-based access control ensures that only authorized users can modify or deploy the application.

2. Secure Storage of Credentials

- Sensitive information such as Docker Hub credentials, API keys, and AWS access details are stored using **environment variables or secrets** (e.g., GitHub Secrets).
- This prevents exposure of confidential data in the source code or pipeline configuration files.
- Secrets are encrypted and accessed only during runtime.

3. Secure Communication

- All communication between system components is secured using protocols like **HTTPS and SSH**.
- Data transmitted between users, servers, and services is encrypted to prevent interception or tampering.

4. Container Security

- Docker images are built using minimal base images to reduce vulnerabilities.
- Only required dependencies are included, minimizing the attack surface.
- Regular updates and patches are applied to ensure security compliance.

5. Infrastructure Security (AWS EC2)

- AWS **Security Groups** act as virtual firewalls, allowing only required ports (e.g., 22 for SSH, 80/443 for HTTP/HTTPS).
- Unnecessary ports and services are disabled to prevent unauthorized access.

6. Code Security and Integrity

- Version control ensures traceability of all code changes.
- CI/CD pipelines validate builds before deployment, reducing the risk of deploying insecure or faulty code.
- Optional code scanning tools can be integrated to detect vulnerabilities.

7. Backup and Recovery

- Source code is safely stored in GitHub with version history.
- In case of failure, previous stable versions can be restored easily.
- Docker images with version tags allow rollback to earlier deployments.

7.2 Performance Aspects

Performance requirements ensure that the system operates efficiently, delivers fast responses, and handles workload effectively.

1. Fast Build and Deployment

- The CI/CD pipeline automates the build and deployment process, significantly reducing manual effort and time.
- Code changes are automatically tested, built, and deployed, ensuring rapid delivery.

2. Efficient Resource Utilization

- Docker containers provide lightweight virtualization, using fewer resources compared to traditional virtual machines.

3. Caching Mechanisms

- Caching is implemented at multiple levels:
  - Browser caching for static assets
  - Dependency caching in CI/CD pipelines

4. Scalability

- The application is designed to be **stateless**, enabling easy scaling.
- Additional instances or containers can be deployed without affecting existing sessions.
- If Kubernetes is used, automatic scaling and load balancing can be achieved.

5. Optimized Application Performance

- Vue.js provides a fast and reactive user interface.
- Static files are optimized during the build process, reducing load time.
- Efficient frontend design ensures smooth user interaction.

6. High Availability

- The system can be redeployed quickly in case of failure.
- Containerized applications ensure consistent behavior across environments.
- Optional load balancing can improve availability in production environments.

7. Monitoring and Logging

- CI/CD tools provide logs for each pipeline stage, helping identify performance issues.
- Application logs on EC2 help monitor runtime behavior.
- These logs assist in debugging and performance optimization.

8. References

The following resources were used for understanding, designing, and implementing the CI/CD pipeline using AWS EC2, Docker, and related technologies:

1. **Amazon Web Services (AWS) Documentation**
   Official documentation for setting up and managing EC2 instances
   https://docs.aws.amazon.com/

2. **Docker Documentation**
   Guide for containerization, Dockerfile creation, and image management
   https://docs.docker.com/

3. **GitHub Actions Documentation**
   Reference for creating CI/CD workflows and automation pipelines
   https://docs.github.com/en/actions

4. **Kubernetes Documentation**
   Documentation for container orchestration and deployment
   https://kubernetes.io/docs/

5. **Vue.js Official Documentation**
   Guide for building and deploying Vue.js applications
   https://vuejs.org/

6. **Jenkins Documentation**
   Reference for CI/CD pipeline setup using Jenkins
   https://www.jenkins.io/doc/